

Getting Started with Stata

Workshop 1 · 75 minutes · No statistics background assumed. No prior Stata assumed.

Setting up

1. Get the files. Go to tinyurl.com/ndpop-stata-1 → the green **Code** button → **Download ZIP**. Then **unzip** it somewhere you can find again.

2. Stata is already installed on the lab workstations — nothing to set up. If you would rather use your own laptop that is fine, as long as Stata is already on it; everything today works the same on Windows and Mac. If you need help installing it, catch me afterwards.

3. Double-click `first_looks.dta` inside the folder you just unzipped. **That is how you open Stata today** — opening the data opens the program, with the data already in it. No path to type and nothing to configure. What that quietly did for you is the subject of 2.9.

Lab workstations are shared machines. Before you leave, copy your work to a USB stick, your email, or a cloud drive. That matters more than usual today: Block 4 is yours to take home.

How this sheet works

Type the monospace lines exactly as printed. Nothing on this sheet has a blank to fill in.

Boxes with a bar down the side are marked *Advanced* — *not for today*. They are real, working code, printed so you have it, with no task attached. Run them after the session; every one of them is also in `block2.do` or `block3.do`.

The three blocks lean on you more and more, on purpose. Block 2 is at the Command window, with every command printed here. Block 3 moves into a do-file: `block3.do` has the code already written, and your job is to run it a line at a time and work out what each line did before you move on. Block 4 is the same again, on your own, afterwards. You are never asked to invent a command from nothing — you are asked to understand one, and then to keep going with nobody next to you.

Most of this sheet tells you what to notice as you go. Three places instead ask you to stop and work something out before reading on — the prediction at 2.5, the failure at 3.3, and the question that closes the session.

Check: lines say what you should see. That is how you check yourself without asking.

Blocks 2 and 3 both use `first_looks.dta`, and nothing you do in Block 2 needs saving. Block 4 imports a CSV instead.

The workflow

what an analysis is made of

COLLECT → INSPECT ↔ CLEAN → ANALYZE → REPORT

COLLECT	not today	— the data was extracted for you
INSPECT	Block 2	in this room
CLEAN	Block 3	in this room
ANALYZE	Block 4	in the file you take home
REPORT	Block 4	in the file you take home

INSPECT and CLEAN are a loop, not two steps in a row. Block 3 is where you run that loop for real.

Four of the five are here today — INSPECT and CLEAN in this room, ANALYZE and REPORT in the Block 4 file you take home.

COLLECT is the one nobody does today. The Google Trends export and the Census extract were prepared before you arrived.

And correlate and regress are two lines each. Knowing *which* two lines, and what they entitle you to say, is the rest of a course and degree. Today is the part that comes before that question — and it is the same regardless of what the question turns out to be. Which is why it is worth learning first, and why Workshop 2 is also entirely on this side of the line.

Block 2 First look at Stata inspect

Work in the Command window (bottom center panel).

2.1 Open it

The data is already open — double-clicking it started Stata. Your first command is just:

`browse`

`browse` opens a spreadsheet view. It is read-only — look, do not edit. Close that window to get back to the Command window.

One row is one state — 50 of them, from the 1980 US Census — and one column is one thing measured about every state.

Hold onto the row question. It comes back in Block 4, and Workshop 2 is built on it.

2.2 The windows

Nothing to type. **Results** is what Stata said. **Command** is where you type. **History** is everything you have run — Block 2 ends by turning it into a file. **Variables** lists what is in the data; double-clicking a name types it for you. **Properties** shows what Stata knows about the variable you have selected.

Use a drop-down menu with the Results window in view. Stata writes out the command it just ran. That is how you find a command name you were about to go searching for.

2.3 Two small experiments

```
Browse
bro

command Browse is unrecognized
r(199);
```

Two lessons. Capitals matter: Stata is case-sensitive, and `Browse` is not a command. And most commands have a legal short form — convenient to type, unkind to whoever reads it later, including you.

You have also now seen what an error looks like. Errors are normal. *Read them.*

2.4 Describing, and the shape of a command

Run these one at a time and watch what changes.

```
describe
describe pop
describe pop-popurban
describe pop*
describe pop??p
```

Every Stata command has the same shape: command, then a `varlist`, then a comma and its options. Only the command itself is compulsory.

`pop-popurban` is a *range* through the dataset's own order, not an alphabetical one. `pop*` is a wildcard; `pop??p` matches one character per `?`.

Advanced — not for today. Ranges only run forwards. `pop` comes after `state` in the dataset, so asking for `pop-state` asks Stata to run backwards. The error is worth seeing once — it is `r(111)`, which you will meet again as *variable not found*.

```
describe state pop-state
```

```
variables out of order
r(111);

codebook pop has_nd
inspect pop has_nd
summarize pop has_nd
```

Three ways of looking at the same variables. `codebook` takes them one at a time and in detail, `inspect` shows you where the values sit and whether any are missing, and `summarize` gives the summary statistics and nothing else.

2.5 Listing rows, and if

```
list state has_nd
list state has_nd if has_nd == 1
```

Check: Indiana, and only Indiana. `==` asks a question; `=` assigns a value. They are not interchangeable.

Predict first. Write down what you think each of the next two returns — then run them.

```
list state has_nd if has_nd != 0
list state has_nd if has_nd > 0
```

Were you right? If not, what is going on?

Stata arbitrarily treats missings as a numeric value greater than any number. This is just something to be aware of.

```
help list
```

2.6 Counting

```
tabulate border
tabulate state border
tabulate border coastal
tabulate border coastal, row column
```

Check: 35 and 15 on the first one. The second fills the screen with 50 rows, which is itself the lesson about when a cross-tabulation stops being useful.

Everything after the comma is an option. That is the last piece of the shape from 2.4.

```
count if border == 1 & coastal == 1
list state if border == 1 & coastal == 1
```

Check: 7 states — Alaska, California, Maine, New Hampshire, New York, Texas, Washington.

2.7 Sorting

Advanced — not for today. Row order. `sort` goes up only; `gsort` takes a minus sign to go down. `in` filters by row *position* and `if` by *value*, so `in` depends on how the data happens to be sorted and `if` never does. The last line gives the five states with the most people over 65 — which is not the same thing as the five oldest states.

```
sort pop
gsort pop65p
gsort -pop65p
list state pop65p in 1/5
```

2.8 Your first do-file

This is the point of the whole block, and it is done by clicking.

1. **History** window → the circular warning icon, which hides the commands that errored → **Copy All**
2. `doedit` — an empty do-file editor opens
3. Paste
4. Save it as `first_do.do`
5. `do first_do.do`

This can turn hours of interactive typing into something you can run again in seconds.

If someone handed you that file tomorrow, they could not follow it. It is a wall of commands with no explanation, including your typos and your dead ends. What is missing is comments — which is exactly what `block3.do` opens with.

2.9 Where has Stata been working all this time?

```
cls
pwd
```

Read what it says. You never told Stata where to work, and it has an answer anyway — because double-clicking a file chose the folder for you. Now watch what happens when nothing chooses. **Close Stata completely**, and start it again from the **Windows Start menu** (or **Spotlight** on a Mac) — not by double-clicking anything.

```
pwd
```

Check: somewhere else entirely — whatever folder Stata happens to start in. Your data is not there, and nothing you type will find it until you say where to look.

So say where. Open your unzipped folder in File Explorer or Finder, click the address bar, and copy the path. Then type `cd "`, paste the path, and close the quote — it will look like `cd "C:/Users/nd_netid/Downloads/stata-onboarding"`. This is the one line on this sheet that is not typed exactly as printed, because only you know where your folder is. Then check it:

```
pwd
```

Check: `pwd` now prints your workshop folder. **Use forward slashes** — they work on Windows and Mac both, and Windows hands you backslashes when you paste, so change them.

Block 3 Your first Stata project clean

Open `block3.do`. The commands are already written. Your job is to run them **a line at a time** — highlight the line and press **Ctrl-D** (Windows) or **Cmd-Shift-D** (Mac) — and work out what each one did before you move on. This sheet numbers the steps and tells you what to look for.

3.1 Where Stata is working

1. Run `pwd`. If it is not your workshop folder, `cd` to it as you did in 2.9. Everything after that is a bare filename.

Advanced — not for today. Two shorthands, and the argument about paths. `.` is where you are; `..` is one level up. A path like `first_looks.dta` breaks the moment you move the file; a path like `C:/Users/you/...` breaks the moment anyone else opens your do-file. Neither is safe on its own, which is why the habit is to `cd` once at the top and use relative names after.

```
cd .
cd ..
pwd
```

3.2 What a do-file starts with

2. Run the four lines under the `use`. They are Stata's three ways of writing a comment, and they demonstrate themselves: `*` comments a whole line, `//` can follow a command on the same line, and `/* */` spans several. Watch which variables `desc state` `// pop` actually describes.

Stata ignores comments; people read them. Put your own name and today's date at the top of `block3.do` — the thing everybody skips.

3.3 Does the data add up?

Population is split by age — `pop1t5`, `pop5_17`, `pop18p`, `pop65p`. Those four should total `pop`.

3. Run the `generate` and `summarize` lines.

Check: the means differ — 4,518,149 against 5,027,652.

Your eyes can catch that. This is harder to catch by eye with bigger datasets.

4. Run the `assert`:

```
assert pop == check_pop
50 contradictions in 50 observations
assertion is false
r(9);
```

This fails on purpose, and it stops the do-file. That halt is the entire point of `assert`: a summary table you skim past is easy to ignore, and a file that refuses to continue is not.

Why did it fail? Look at what each of the four age variables actually covers — and work it out before you run the next line.

5. Then carry on. The `replace` redefines `check_pop` using only the groups that do not overlap, and the `assert` runs again.

Check: silence. `assert` prints nothing at all when it passes.

6. Run the rest of the section: the same check from the other side, `pop18p` minus `pop65p`. **That name is wrong** — the group runs 18 to 64, not 18 to 65 — so the next line `renames` it, then builds `check2_pop`, asserts it, and drops both check variables.

3.4 Functions

Advanced — not for today. Stata evaluates maths expressions, and ships a library of functions for numbers, strings and dates. Nobody memorises the list — knowing it exists, and how to search it, is the skill.

```
generate pop_m = pop / 1000000
replace pop_m = round(pop_m, .1)
help functions
```

3.5 Stata remembers more than it shows

Advanced — not for today. Every command leaves its results in memory, and `return list` shows them. They are overwritten by the next command that returns anything, so use them immediately or store them first. `egen` reaches the same answer by a different route — worth knowing, because it usually does.

```
summarize pop, detail
return list
generate big_state = (pop >= r(p90))
list state pop if big_state == 1

egen big_state2 = pctlile(pop), p(90)
replace big_state2 = pop >= big_state2
assert big_state == big_state2
drop big_state2
```

3.6 Text

Advanced — not for today. Advanced string work. Five state names are stored short — N. Carolina, N. Dakota, S. Carolina, S. Dakota, W. Virginia. `regexm()` asks whether a string matches a pattern, and `"\."` means “contains a full stop”. You are not expected to write regular expressions; recognising one is enough. `inlist()` is the readable way to say “any of these”, and writing it instead as `if state2 == "SC" | "SD"` reads like English but is not valid — “SD” on its own is not a test, and Stata answers `type mismatch, r(109)`. `preserve` and `restore` because that `replace` overwrites `state`, and you want your data back afterwards.

```
preserve
list state state2 if regexm(state, "\.")
generate state3 = substr(state, 4, strlen(state)) if regexm(state, "\.")
replace state = "South" if inlist(state2, "SC", "SD")
restore
```

3.7 Labels

Three kinds, broadest to narrowest: the dataset, a variable, and the values inside a variable.

7. Run the `label data` and `label variable` lines — a title for the dataset, a readable name for `coastal`.

8. Value labels work differently: you define a named set once, then apply it to as many variables as you like. Run the `tabulate`, then the `label define` and `label values`, then the same `tabulate` again.

Check: same numbers, readable table.

The labels changed; the data did not. Labels sit on top of the values — the numbers underneath have not moved, which is the more interesting half of it.

Advanced — not for today. `region` arrived with a value label already attached, called `cenreg`, and `, modify` edits an existing set in place rather than defining a new one. `nolabel` is singular — `nolabels` gives option `nolabels` not allowed, `r(198)`. Note that this changes how `region` prints for the rest of your session.

```
tabulate region
label define cenreg 1 "Northeast" 2 "Midwest" 3 "South" 4 "West", modify
tabulate region
tabulate region, nolabel
```

3.8 Groups

9. Run the first four lines and look at what they made. `_n` is *this row's number* and `_N` is *the total row count*, repeated identically on every row. Then they are dropped again.

10. Now run the `bysort region:` lines. Putting `bysort region:` in front of a command makes Stata run it once per region — and that changes what `_n` and `_N` refer to. The last line lists one row per region instead of all 50.

Check: four rows — 9, 12, 16 and 13 states. The totals print in scientific notation, because they are raw counts of people. `format region_pop %15.0fc` makes them readable.

Inside `bysort region:`, `_N` counts the rows in *that region*, not in the dataset — and if `nval == 1` keeps only the first row of each region, which is how four rows come back instead of fifty.

This is the idea Workshop 2's `collapse` is built on.

3.9 Log files

The do-file is what you asked. The log is what Stata answered.

11. Run the log section. It opens `test_log.txt`, prints two things to it, and closes it again.

Check: `test_log.txt` appears in your folder and opens as readable text.

Three small protections are doing real work in those two lines. `capture` stops a failed `log close` from halting the whole file, so you can run it twice. `,` `text` keeps the log readable — without it you get a screenful of `{smcl}` markup. `,` `replace` lets the second run overwrite the first.

12. Run the `save`. Your cleaned data becomes `first_looks_clean.dta`.

Block 4 First output from raw input yours to take home

We do not work through Block 4 in the room. It closes the session instead: one figure on the screen, and `block4.do` is the file that makes it. It is written to be followed on your own, from cold — nothing in it depends on anything done during the session.

It takes a genuinely raw file — a Google Trends export of search interest for Notre Dame, MIT and Clemson, 272 monthly rows from 2004 to 2026 — and carries it all the way to a figure you could put in a paper. Seven steps:

1. **Import** the CSV and inspect what arrived. *272 observations, 4 variables.*
2. **Fix the dates.** They arrive as text and are useless until converted. *It should read 01jan2004, not 16071.*
3. **Rename and label.** One variable imports as `massachusettsinstituteoftechnolo`, truncated at 32 characters.
4. **Declare it a time series** with `tsset`, or the line graphs will not draw. *2004m1 to 2026m8, delta: 1 month.*
5. **Analyze** — correlate, then regress.
6. **Draw it** — scatter, `twoway`, `tsline`, working up from the plain version to the layered one.
7. **Export** a figure. *A .png appears in your folder.*

In Block 2, one row was one state. What is one row here?

The commands are written out, and most carry a **help:** line naming the help file behind them. Open those. Read the **Syntax** block at the top and the **Examples** at the bottom; the middle is reference you can skip until you need it. That is the actual skill — nobody memorises Stata, they know what to look up.

Block 5 Where to go next

Three things to look up: macros (`help macro`) · loops (`help foreach`) · user-written commands (`ssc install`)

Getting unstuck, in order: `help commandname` → search keyword → the internet, pasting the exact error text with the `r(###)` included → a person.

People who will help you:

- **ND Pop Stata Helpdesk** — drop in, no appointment. **Mon–Thu 5–7pm, 4020D Jenkins Nanovic Halls.** Bring your laptop. populationanalytics.nd.edu/resources/stata-support-services
- **Navari Family Center for Digital Scholarship** — Hesburgh Library, 2nd floor NE corner. Consultations on data analysis, visualisation, GIS and text mining. cds@nd.edu

Online: the `help` command, which is the full manuals, already installed · stata.com/support · blog.stata.com · stats.oarc.ucla.edu/stata · stata.com/flyers/stata-cheat-sheets — free PDFs, including one for syntax.

AI chatbots are useful for finding a command name or decoding an error. They also invent options that do not exist, fluently and confidently. **Never run a Stata command from a chatbot until you understand it independently.**

A book, for later: Cameron & Trivedi, *Microeconometrics Using Stata*, 2nd ed. (Stata Press, 2022). A graduate text, well beyond today, but its early chapters on syntax and data management cover the ground you started here. Check the library.

Before you log out: copy your do-files off this machine.

Afterwards Start with `block4.do` — the take-home. `block2.do` and `block3.do` are today written out in full, and `README.md` lists the rest. All of it stays on github.com/cassandramerriitt/ndpop-stata-workshops. Your own do-files do not — save those yourself.